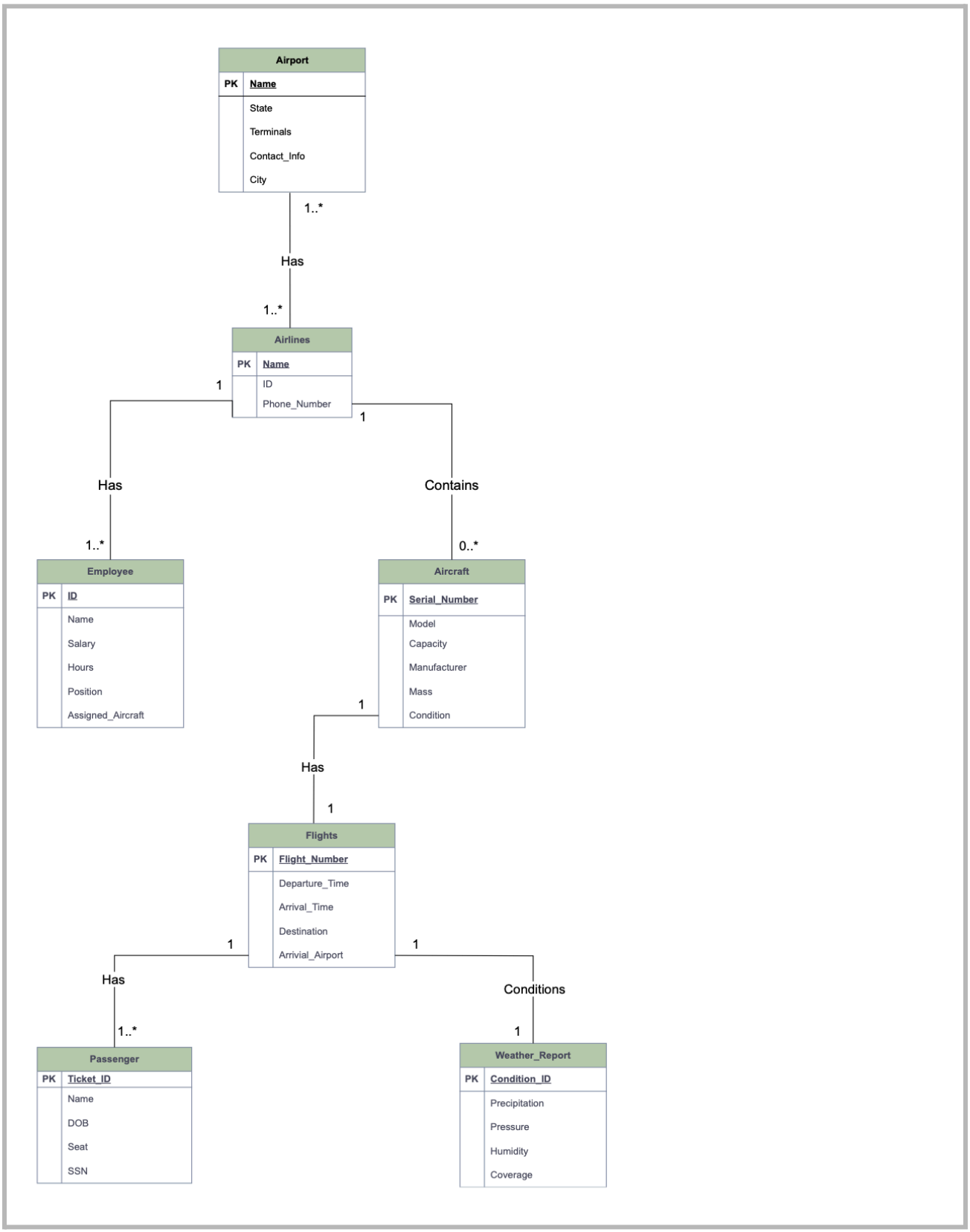
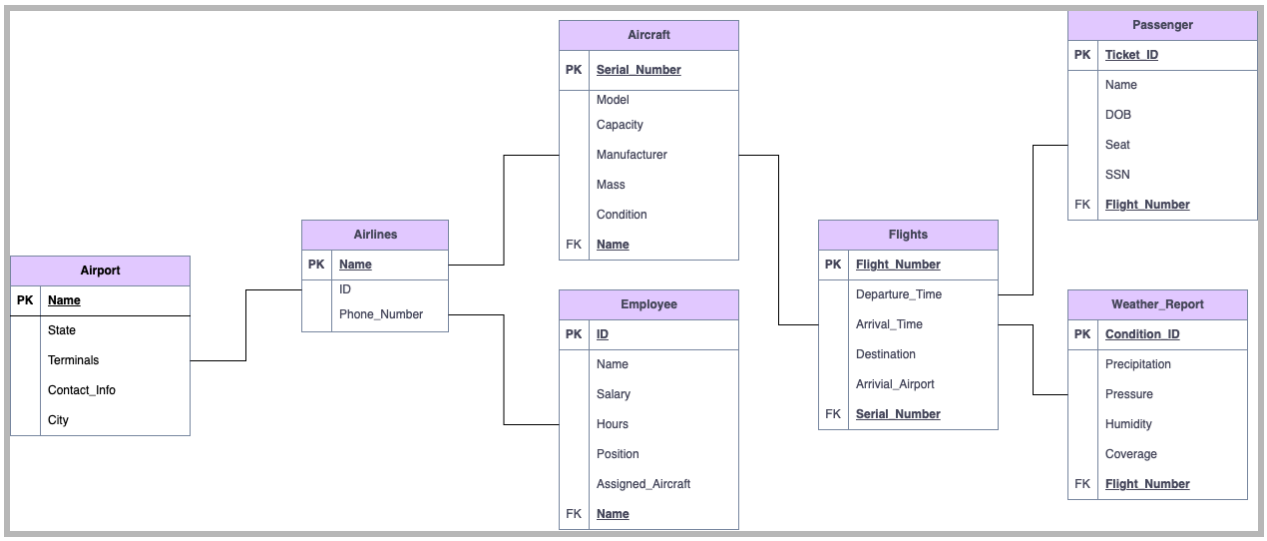
Entity Relationship Diagram



Relational Model and Normalization



Functional Dependencies (Primary Keys in Bold)

Airport - **Name** - State, Terminals, Contact_Info, City

Airlines - **Brand_Name** - ID, Number

Aircraft - **Serial_Number** - Model, Capacity, Manufacturer, Mass, Condition, Name (FK)

Employee - **ID** - Name, Salary, Hours, Position, Name (FK)

Flights - **Flight_Number** - Departure_Time, Arrival_Time, Destination, Arrival_Airport, Serial (FK)

Passenger - **Ticket_ID** - Name, DOB, Seat, SSN, Flight_Number (FK)

Weather - **Condition_ID** - Precipitation, Pressure, Humidity, Coverage, Flight_Number (FK)

Normalization

1NF - Every attribute is a singular/atomic value and each table has a primary key so it is in 1st Normal Form.

2NF - Each table has a single primary key for dependency so there is no partial dependencies and every table is in 2nd Normal Form.

3NF - Each table has single atomic values, each attribute relies fully on the primary key and no partial dependencies and no transitive dependencies, no derived attributes from non primary keys/other attributes. This database is in 3rd Normal Form.

SQL DDL for Tables

```sql
CREATE TABLE AIRPORT(
   Name VARCHAR(255) NOT NULL PRIMARY KEY,
   State VARCHAR(255),
   Terminals INT,
   Contact_Info VARCHAR(255),
   City VARCHAR(255)
);

CREATE TABLE AIRLINES(
   Brand_Name VARCHAR(255) NOT NULL PRIMARY KEY,
   ID INT,
   Phone_Number INT
);

CREATE TABLE AIRCRAFT(
   Serial_Number INT NOT NULL PRIMARY KEY,
   Model VARCHAR(255),
   Capacity INT,
   Manufacturer VARCHAR(255),
   Mass NUMERIC,
   Condition VARCHAR(255),
   Brand_Name VARCHAR(255) FOREIGN KEY REFERENCES AIRLINES(Brand_Name)
);

CREATE TABLE EMPLOYEE(
   ID INT NOT NULL PRIMARY KEY,
   Name VARCHAR(255),
   Salary NUMERIC,
   Hours INT,
```

```sql
    Position VARCHAR(255),
    Brand_Name VARCHAR(255) FOREIGN KEY REFERENCES AIRLINES(Brand_Name)
);

CREATE TABLE FLIGHTS(
    Flight_Number INT NOT NULL PRIMARY KEY,
    Departure_Time TIMESTAMP,
    Arrival_Time DATE,
    Destination VARCHAR(255),
    Arrival_Airport VARCHAR(255),
    Serial_Number INT FOREIGN KEY REFERENCES AIRCRAFT(Serial_Number)
);

CREATE TABLE Passenger(
    Ticket_ID INT NOT NULL PRIMARY KEY,
    Name VARCHAR(255),
    DOB DATE,
    Seat INT,
    Flight_Number INT FOREIGN KEY REFERENCES FLIGHTS(Flight_Number)
);

CREATE TABLE Weather(
    Condition VARCHAR(255) NOT NULL PRIMARY KEY,
    Precipitation NUMERIC,
    Pressure NUMERIC,
    Flight_Number INT FOREIGN KEY REFERENCES FLIGHTS(Flight_Number)
);
```

Case Scenarios

Case 1:

A common question that can be asked is about the weather for the scheduled flights. A passenger or airlines administration would like to know which flights may have high precipitation, pressure or poor weather conditions. This is useful to determine whether a flight should be cancelled or delayed.

Query will find flights that have a high precipitation or pressure (measured per hour and millibars) or a condition not labeled as 'Good' and ordered by there flight number.

Query:

```
SELECT *
FROM Weather
WHERE Precipitation > 2000 OR Pressure > 1000 OR Condition != 'Good'
ORDER BY Flight_Number;
```

Case 2:

Information about employees can be very useful for determining how to allocate financial resources and analyze performance. Airlines will have different averages from one another. Management may ask questions such as, which airline has the highest average employee salaries? What are the average hours worked?

Query will find the average hours worked and salaries of the employees and group them by the airlines they are associated with, in essence it shows the average employee performance/details of each airlines.

Query:

```
SELECT AVG(Salary) AS Average_Salary, AVG(Hours) AS
Average_Hours_Worked, Brand_Name
FROM EMPLOYEE
GROUP BY Brand_Name;
```

Case 3:

We want to gather information about flights that are scheduled on Christmas Day. This information is useful for analyzing airport and airline traffic on the holidays which can be beneficial for management to track and optimize resources during these dates.

Query will find information such as the flight number, its destination, and the specifications of the aircraft for the flight.

Query:

```
SELECT FLIGHTS.Flight_Number, FLIGHTS.Destination,
FLIGHTS.Serial_Number, AIRCRAFT.Model,
AIRCRAFT.Condition, AIRCRAFT.Brand_Name
FROM FLIGHTS
INNER JOIN AIRCRAFT ON FLIGHTS.Serial_Number = AIRCRAFT.Serial_Number
WHERE FLIGHTS.Departure_Time = '2025-12-25';
```

Case 4:

A very common scenario when dealing with flight cancellation is assessing refunds to the passengers. In a specific case regarding aircraft quality, we will need to find information about passengers on a certain scheduled flight which is associated with an aircraft of unsafe flight condition.

Query will find the passengers Ticket ID, the flight number and several aircraft specifications such as serial number, model, condition and the airlines it is associated with. It will display all the passengers who will be apart of a flight that is associated with a faulty aircraft.

Query:

```
SELECT Passenger.Ticket_ID, Passenger.Flight_Number,
FLIGHTS.Serial_Number,
AIRCRAFT.Model, AIRCRAFT.Condition, AIRCRAFT.Brand_Name
FROM Passenger
INNER JOIN FLIGHTS ON Passenger.Flight_Number = FLIGHTS.Flight_Number
INNER JOIN AIRCRAFT ON FLIGHTS.Serial_Number = AIRCRAFT.Serial_Number
WHERE Condition != 'Safe';
```

Conclusion

Our Experience:

This project taught us how to create relational models between entities and implement them into a database which is useful for real world database management applications such as Airport Management. The difficult parts was deciding on what the relationships should be as it could have caused conflicts or confusion when describing how entities are interconnected. It gave us a strong foundation into databases, relational models and practical usage of SQL.